

Process Environment in Linux and Memory Layout for a Process

Process Environment Variables

- Environment Variables allows Linux processes to know the context they're running in.
- The environment variables may store information about the location of system libraries, user profile settings, or temporary file directories.

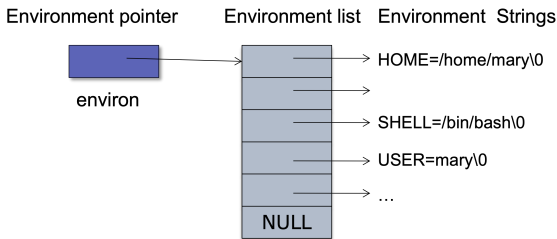
E.g., to see the environment list for a process id 1234:

```
cat /proc/1234/environ
```

```
LANG=en_US.UTF-8
USER=mary
LOGNAME=mary
HOME=/home/mary
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
SHELL=/bin/sh
TERM=xterm-256color
XDG_SESSION_ID=193
XDG_RUNTIME_DIR=/run/user/1002
XDG_SESSION_TYPE=tty
XDG_SESSION_CLASS=user
MOTD_SHOWN=pam
SSH_CLIENT=23.66.16.13 59035 22
SSH_TTY=/dev/pts/0
```

Environment List

- Each program is passed (silently) an environment list.
- Like the argument list, the environment list is an array of pointer to c-string.
- Each pointer contains the address of null terminated character string.
- The global variable `environ` contains the address of the array pointers.



Code Example for Using The Global Variable: environ

```
/* environ1.c: display environment strings */
#include <stdio.h>
extern char **environ;
int main(int argc, char *argv[]) {
    char **p = environ;
    while (*p != NULL) {
        printf("%s \n", *p);
        p++;
    }
    return 0;
}
```

```
/* environ2.c: display environment strings with corresponding address */
#include <stdio.h>
int main(int argc, char *argv[], char *envp[]) {
    char **p = envp;
    while (*p != NULL){
        printf("%s (%p)\n", *p, *p);
        p++;
    }
    return 0;
}
```

Environment Variable Functions

- An environment variable is a named object that contains information used by one or more applications.
- ANSI C defines several functions for getting/setting values from the environment.

```
#include <stdlib.h>

// return pointer to the value associated with the name
char *getenv(const char *name)

/* If name doesn't exist, it is inserted with the given value.
   If name does exist, the argument rewrite is tested;
       if rewrite is zero, the name is not reset,
       otherwise it is reset to the given value. */
int setenv (const char *name, const char *value, int rewrite)

/* put a new environment variable during the process running */
int putenv (const char *name)

/* deletes all instances of the variable
   pointed to by name from the list. */
int unsetenv(const char *name)
```

Code Example for Environment Variable Functions

```
#include <stdlib.h>
#include <stdio.h>

int main() {

    printf("HOME=%s\n", getenv("HOME"));
    printf("PATH=%s\n", getenv("PATH"));

    char *env1 = getenv("TEST11");
    printf("TEST11=%s\n", env1); //show current env variable

    setenv("TEST11", "abcd", 1); //reset it
    env1 = getenv("TEST11");
    printf("TEST11=%s\n", env1);

    putenv("MYENV=mary");
    printf("MYENV=%s\n", getenv("MYENV"));
    return 0;
}
```

- For `setenv()`, `putenv()`, `unsetenv()`, it returns 0 on success; -1 on error.
- The functions `setenv()`, `putenv()`, `unsetenv()` failed if the `name` is a NULL pointer, points to an empty string, or points to a string containing an "=" character.
- The function `putenv()` failed if `name` is a NULL pointer or `name` is without an "=" character.
- The functions `setenv()`, `putenv()`, `unsetenv()` failed if it was unable to allocate memory for the environment.

Memory Layout of for a Process

A C program is composed of five components.

Text (Code) Segment : The machine instruction set, since it might be sharable, only one copy need to be in the memory.

Initialized data Segment : variables that are initialized in the program. E.g.

```
int max = 5;
```

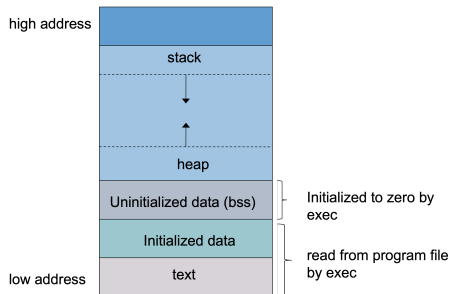
Uninitialized data Segment (bss) : variables that are not initiated.

Note: the *bss* stands for block starting symbol. It is the portion of an object file, executable, or assembly language code that contains statically allocated variables that are declared but have not been assigned a value yet. It is often referred to as the “bss section” or “bss segment”.

Stack : saved temporary variables when a function is called, including the caller’s environment value such as return address.

Heap : used for dynamic memory allocation.

Memory Layout for a Process (Cont')



The `size` command returns the size in bytes of the text, data, and uninitialized data(bss) segments. The `dec` is the total addition of text, data, and bss values; same for `hex`.
E.g.:

```
size usr/bin/gcc /bin/sh
```

text	data	bss	dec	hex	file name
115651	1744	1140	118535	1cf07	/usr/bin/gcc
485881	8936	25360	520177	7eff1	/bin/sh

Example Program for Examining Memory Address

```
#include<stdio.h>
#include<stdlib.h>

int global;

int main(int argc, char *argv[]){

    int var;
    printf("argc at: 0x%12lx\n", (unsigned long) &argc);
    printf("argv at: 0x%12lx\n", (unsigned long) &argv);
    printf("local var at: 0x%12lx\n", (unsigned long) &var);
    printf("global var at: 0x%12lx\n", (unsigned long) &global);
    printf("main at: 0x%12lx\n", (unsigned long) &main);
    return 0;
}
```

Dynamic Memory Allocation Functions

There are three functions for memory allocation by C. (located in the heap)

malloc() : allocates a specified number of bytes (size of a certain datatype) of memory. It doesn't initialize the allocated memory (which means the programmer needs to do initialization).

```
void *malloc(size_t size)
```

calloc() : (continuously) allocates space for a specified number of objects, each for a specified size (of a certain datatype). The space is initialized to all 0 bits.

```
void *calloc(size_t nitems, size_t size)
```

realloc() : change the size of a previously allocated area.

```
void *realloc(void *ptr, size_t size)
```

free() : deallocates the space pointed to by ptr.

```
free(ptr)
```

Dynamic Memory Allocation - Code Example 1

```
/* malloc.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){
    typedef struct {
        int age;
        char name[20];
    } data;

    data *bob;
    bob = (data*) malloc( sizeof(data) );
    if( bob != NULL ) {
        bob->age = 22;
        strcpy( bob->name, "Mary" );
        printf( "%s is %d years old\n", bob->name, bob->age );
    }
    free( bob );
    return 0;
}
```

Dynamic Memory Allocation - Code Example 2

```
/* calloc.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){
    typedef struct data_type {
        int age;
        char name[20];
    } data;
    data *bob;
    bob = (data*) calloc( 2, sizeof(data) );
    if( bob != NULL ) {
        bob[0].age = 22;
        strcpy( bob[0].name, "Mary" );
        bob[1].age = 25;
        strcpy(bob[1].name, "Chris");
        printf( "%s is %d years old\n", bob[0].name, bob[0].age );
        printf( "%s is %d years old\n", bob[1].name, bob[1].age );
    }
    free( bob );
    return 0;
}
```

Dynamic Memory Allocation - Code Example 3

```
/* calloc1.c */
#include <stdio.h>
#include <stdlib.h>

int main(){
    int i, n;
    int *a;

    printf("Number of elements to be entered:");
    scanf("%d",&n);

    a = (int*)calloc(n, sizeof(int));
    printf("Enter %d numbers:\n",n);
    for( i=0 ; i < n ; i++ ) {
        scanf("%d",&a[i]);
    }
    printf("The numbers entered are: ");
    for( i=0 ; i < n ; i++ )
        printf("%d ",a[i]);
    free(a);
    return(0);
}
```

Dynamic Memory Allocation - Code Example 4

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct {
    int age;
    char name[20];
} data;

int main(){
    int i, n;
    char iname[20];
    int iage;
    data *list;
    printf("Number of students to be entered in the list:");
    scanf("%d",&n);
    list = (data*) calloc( n,sizeof(data) );
    for (i=0; i < n; i++){
        printf("Enter name: \n");
        scanf("%s",iname);
        strcpy (list[i].name, iname);
        printf ("Enter age: \n");
        scanf("%d", &iage);
        list[i].age =iage;
    }
    printf ("YOUR STUDENT LIST\n");
    for (i=0; i < n; i++){
        printf("%s    %d \n",list[i].name, list[i].age);
    }
    free( list );
    return 0;
}
```